


# PRÁCTICA GUIADA: PROYECTO “SERVICIOS TÉCNICOS”

*Preparación del Backend y conexión Python–MySQL*

## 1. Propósito

Construir paso a paso la primera parte del proyecto “Servicios Técnicos”, preparando el entorno de desarrollo, creando el backend con Python y Flask, configurando la conexión con MySQL y comprobando que Python pueda acceder a la base de datos técnico. En esta etapa todavía no se desarrolla React ni la API de login.

## 2. Objetivos

- Configurar Python y un entorno virtual para el backend.
- Instalar Flask, mysql-connector-python y Flask-CORS.
- Crear la base de datos tecnico y la tabla usuario.
- Organizar el backend en módulos.
- Separar la configuración de conexión en conexion.py.
- Comprobar la conexión Python → MySQL.

## 3. Arquitectura del proyecto

```
React + JavaScript + CSS
        │ HTTP / JSON
        ▼
Python + Flask (API REST)
        │ SQL
        ▼
MySQL → base de datos tecnico
```

Regla fundamental: React no se conectará directamente a MySQL. La comunicación será React → API Flask → MySQL.

## 4. Herramientas necesarias

- Python
- Node.js y npm
- Git
- MySQL
- Visual Studio Code




Comprueba las versiones:

```
python --version

pip --version

node --version

npm --version

git --version
```

## 5. Crear el proyecto

Crea y abre en Visual Studio Code la carpeta:

```
Practica_servicios_tecnicos
```

Dentro crea:

```
Practica_servicios_tecnicos/
├── backend/
├── frontend/
└── database/
```

## 6. Crear la base de datos MySQL

Abre MySQL Workbench, phpMyAdmin o la consola de MySQL.

```
CREATE DATABASE tecnico;
USE tecnico;
```

Crea la tabla:

```
CREATE TABLE usuario (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL
);
```

Agrega usuarios de prueba:

```
INSERT INTO usuario (username, password)
VALUES ('admin', '12345');

INSERT INTO usuario (username, password)
VALUES ('alumno', '2026');
```




```
SELECT * FROM usuario;
```

Nota: almacenar contraseñas en texto plano es únicamente para esta práctica introductoria. En una aplicación real se deben utilizar contraseñas con hash.

## 7. Guardar el script SQL

Dentro de database crea:

```
database/tecnico.sql
```

Guarda allí las instrucciones utilizadas para crear la base y la tabla.

## 8. Crear el backend

En la terminal entra a backend:

```
cd backend
```

## 9. Crear el entorno virtual

No crees venv manualmente. Ejecuta:

```
python -m venv venv
```

Python creará automáticamente:

```
backend/venv/
```

## 10. Activar el entorno virtual

En Windows:

```
venv\Scripts\activate
```

Debe aparecer algo parecido a:

```
(venv) F:\...\Practica_servicios_tecnicos\backend>
```

## 11. Instalar dependencias

```
pip install flask

pip install mysql-connector-python

pip install flask-cors
```




Verifica MySQL Connector:

```
pip show mysql-connector-python

python -c "import mysql.connector; print('MySQL Connector funciona')"
```

## 12. Crear requirements.txt

```
pip freeze > requirements.txt
```

## 13. Seleccionar el intérprete de Python en VS Code

1. Presiona Ctrl + Shift + P.
2. Busca Python: Select Interpreter.
3. Selecciona el Python de backend\venv\Scripts\python.exe.

Si VS Code utiliza otro entorno, puede marcar mysql.connector como no resuelto aunque el paquete esté instalado.

## 14. Crear la estructura del backend

```
backend/
├── venv/                          ← creado por Python
├── requirements.txt               ← generado con pip
└── app/                            ← creada manualmente
    ├── __init__.py
    ├── main.py
    ├── probar_conexion.py
    ├── config/
    │   ├── __init__.py
    │   └── conexion.py
    ├── routes/
    │   ├── __init__.py
    │   └── login.py
    └── services/
```

Los archivos __init__.py pueden estar vacíos. venv no se crea manualmente.




## 15. Función de cada elemento

| Elemento | Función |
|---|---|
| venv | Entorno virtual de Python. |
| requirements.txt | Dependencias del backend. |
| app | Paquete principal de Flask. |
| main.py | Punto de entrada del backend. |
| config | Configuraciones. |
| conexion.py | Conexión reutilizable con MySQL. |
| routes | Endpoints y rutas de las APIs. |
| login.py | Posteriormente contendrá la API de login. |
| services | Lógica de negocio que se incorporará después. |

## 16. Crear conexion.py

En app/config/conexion.py escribe:

```
import mysql.connector
from mysql.connector import Error

HOST = "localhost"
USER = "root"
PASSWORD = ""
DATABASE = "tecnico"

def conectar():
    try:
        conexion = mysql.connector.connect(
            host=HOST,
            user=USER,
            password=PASSWORD,
            database=DATABASE
        )
        if conexion.is_connected():
            print("Conexión exitosa a la base de datos")
            return conexion
    except Error as e:
        print(f"Error al conectar a la base de datos: {e}")
    return None
```

Si root tiene contraseña, modifica PASSWORD.




## 17. ¿Por qué separar conexion.py?

Centralizar la conexión permite que todos los módulos reutilicen conectar(). Si cambia el servidor, usuario, contraseña o base de datos, se modifica un solo archivo.

```
from app.config.conexion import conectar
```

## 18. Crear probar_conexion.py

En app/probar_conexion.py escribe:

```
from config.conexion import conectar

conexion = conectar()

if conexion:
    print("==============================")
    print("CONEXIÓN EXITOSA")
    print("Base de datos: tecnico")
    print("==============================")
    conexion.close()
else:
    print("==============================")
    print("ERROR DE CONEXIÓN")
    print("==============================")
```

## 19. Ejecutar la prueba

Desde la carpeta backend y con (venv) activo:

```
python app/probar_conexion.py
```

Resultado esperado:

```
Conexión exitosa a la base de datos
==============================
CONEXIÓN EXITOSA
Base de datos: tecnico
==============================
```




## 20. Solución de problemas

| Mensaje | Solución probable |
|---|---|
| Import mysql.connector could not be resolved | Seleccionar backend\venv como intérprete e instalar mysql-connector-python. |
| ModuleNotFoundError: No module named 'config' | Revisar la estructura y ejecutar desde backend con python app/probar_conexion.py. |
| Access denied | Revisar usuario y contraseña de MySQL. |
| Unknown database 'tecnico' | Crear tecnico o corregir el nombre. |
| Can't connect to MySQL server | Comprobar que MySQL esté iniciado. |

## 21. Evidencias

- Versiones de Python, Node.js, npm y Git.
- Base de datos tecnico y tabla usuario.
- Estructura del proyecto en VS Code.
- Entorno virtual activo.
- Instalación de mysql-connector-python.
- Contenido de conexion.py.
- Ejecución de probar_conexion.py mostrando CONEXIÓN EXITOSA.

## 22. Preguntas de reflexión

4. ¿Qué ventaja tiene utilizar un entorno virtual?

Respuesta: _____

_____

5. ¿Por qué conexion.py está separado?

Respuesta: _____

_____

6. ¿Qué función cumple mysql-connector-python?

Respuesta: _____

_____




7. ¿Por qué React no debe conectarse directamente a MySQL?

Respuesta: _____

_____

8. ¿Qué información contiene tecnico.usuario?

Respuesta: _____

_____

9. ¿Qué sucedería si MySQL está apagado?

Respuesta: _____

_____

10. ¿Qué diferencia existe entre el repositorio local y la base de datos?

Respuesta: _____

_____